1. Análisis del proyecto y entidades

El proyecto cuenta con 10 entidades (roles, usuarios, categorías, productos, carritos, carrito_items, lista_deseos, órdenes, orden_detalle, logs). Esta especificación diseña la API REST completa para las cuatro entidades que ya cuentan con un panel administrativo CRUD implementado en el proyecto — **Rol, Usuario, Producto y Log** — ya que son las que concentran la lógica de creación, edición, borrado lógico/físico y restauración trabajada en las actividades 2.4 a 2.6. Las entidades restantes (Categoría, Carrito, CarritoItem, ListaDeseos, Orden, OrdenDetalle) seguirían exactamente el mismo patrón de diseño documentado aquí, cambiando únicamente sus campos propios.

Entidad	Descripción	Tipo de recurso
Rol	Catálogo de roles de usuario (Administrador, Cliente)	CRUD completo
Usuario	Cuentas del sistema, con foto de perfil y rol asociado	CRUD completo
Producto	Artículos de segunda mano publicados por un usuario	CRUD completo
Log	Bitácora de acciones críticas del sistema	Solo lectura y gestión de ciclo de vida (no se crea ni edita manualmente)

2. Definición de los recursos

Cada entidad principal se publica como un recurso independiente, identificado por su propia URI base bajo el prefijo `/api/admin`, reservado para el área administrativa.

Recurso	URI base	Descripción
Roles	`/api/admin/roles`	Catálogo de roles del sistema
Usuarios	`/api/admin/usuarios`	Cuentas de usuario registradas
Productos	`/api/admin/productos`	Artículos de segunda mano publicados
Logs	`/api/admin/logs`	Bitácora de acciones del sistema

Recurso	URI base	Descripción
Autenticación	`/api/admin/auth`	Inicio/cierre de sesión y usuario autenticado (no es un recurso CRUD, ver sección 9)

3-4. Diseño de endpoints y métodos HTTP

Los métodos mínimos requeridos (GET, POST, PUT, PATCH, DELETE) se emplean según la operación: PUT para reemplazo completo del registro, PATCH para actualización parcial y para cambios de estado (restaurar), y DELETE tanto para el borrado lógico como para el borrado físico (distinguidos por la ruta).

Roles, Usuarios y Productos (CRUD completo)

Operación	Endpoint	Método
Listar registros	`/api/admin/{recurso}`	GET
Consultar un registro	`/api/admin/{recurso}/{id}`	GET
Crear un registro	`/api/admin/{recurso}`	POST
Actualizar un registro (completo)	`/api/admin/{recurso}/{id}`	PUT
Actualizar un registro (parcial)	`/api/admin/{recurso}/{id}`	PATCH
Eliminar lógicamente	`/api/admin/{recurso}/{id}`	DELETE
Listar eliminados (papelera)	`/api/admin/{recurso}/papelera`	GET
Restaurar un registro	`/api/admin/{recurso}/{id}/restaurar`	PATCH
Eliminar físicamente	`/api/admin/{recurso}/{id}/forzar`	DELETE

`{recurso}` = `roles`, `usuarios` o `productos` según el módulo.

Logs (solo lectura y ciclo de vida, sin creación/edición manual)

Operación	Endpoint	Método
Listar registros	`/api/admin/logs`	GET

Operación	Endpoint	Método
Consultar un registro	`/api/admin/logs/{id}`	GET
Eliminar lógicamente	`/api/admin/logs/{id}`	DELETE
Listar eliminados (papelera)	`/api/admin/logs/papelera`	GET
Restaurar un registro	`/api/admin/logs/{id}/restaurar`	PATCH
Eliminar físicamente	`/api/admin/logs/{id}/forzar`	DELETE

5. Especificación de las solicitudes

Se documenta a detalle el recurso **Producto** (el más completo: incluye llaves foráneas y carga de imagen); Rol y Usuario siguen la misma estructura de endpoints, cambiando solo los campos del cuerpo (tabla de campos al final de la sección).

GET /api/admin/productos

- **Descripción:** devuelve el listado paginado de productos activos.
- **Parámetros de URL:** ninguno.
- **Query params:** `page` (int, página actual), `per_page` (int, registros por página, por defecto 5), `categoria_id` (int, filtra por categoría), `buscar` (string, busca por nombre).
- **Headers:** `Authorization: Bearer {token}`, `Accept: application/json`.
- **Request body:** no aplica.

GET /api/admin/productos/{id}

- **Descripción:** devuelve el detalle de un producto específico.
- **Parámetros de URL:** `id` (int, requerido, identificador del producto).
- **Query params:** ninguno.
- **Headers:** `Authorization: Bearer {token}`, `Accept: application/json`.
- **Request body:** no aplica.

POST /api/admin/productos

- **Descripción:** crea un nuevo producto.

- **Parámetros de URL:** ninguno.
- **Query params:** ninguno.
- **Headers:** `Authorization: Bearer {token}`, `Content-Type: multipart/form-data` (por la imagen).
- **Request body:** `nombre` (string, requerido, máx. 100), `descripcion` (string, requerido), `precio` (numeric, requerido, mín. 0), `categoria_id` (int, requerido, debe existir), `usuario_id` (int, requerido, debe existir), `imagen` (file, opcional, jpg/png, máx. 2MB).

PUT /api/admin/productos/{id}

- **Descripción:** reemplaza por completo los datos de un producto existente.
- **Parámetros de URL:** `id` (int, requerido).
- **Headers:** `Authorization: Bearer {token}`, `Content-Type: multipart/form-data`.
- **Request body:** los mismos campos que en POST, todos requeridos salvo `imagen` (opcional; si no se envía se conserva la existente).

PATCH /api/admin/productos/{id}

- **Descripción:** actualiza uno o varios campos del producto sin requerir el resto.
- **Parámetros de URL:** `id` (int, requerido).
- **Headers:** `Authorization: Bearer {token}`, `Content-Type: multipart/form-data`.
- **Request body:** cualquier subconjunto de `nombre`, `descripcion`, `precio`, `categoria_id`, `imagen`.

DELETE /api/admin/productos/{id}

- **Descripción:** elimina lógicamente el producto (soft delete).
- **Parámetros de URL:** `id` (int, requerido).
- **Headers:** `Authorization: Bearer {token}`.
- **Request body:** no aplica.

GET /api/admin/productos/papelera

- **Descripción:** lista los productos eliminados lógicamente.
- **Query params:** `page`, `per_page`.
- **Headers:** `Authorization: Bearer {token}`.

PATCH /api/admin/productos/{id}/restaurar

- **Descripción:** restaura un producto eliminado lógicamente.
- **Parámetros de URL:** `id` (int, requerido).
- **Headers:** `Authorization: Bearer {token}`.
- **Request body:** no aplica.

DELETE /api/admin/productos/{id}/forzar

- **Descripción:** elimina el producto de forma permanente (solo si no tiene relaciones dependientes en carrito_items, lista_deseos u orden_detalle).
- **Parámetros de URL:** `id` (int, requerido).
- **Headers:** `Authorization: Bearer {token}`.
- **Request body:** no aplica.

Campos de cuerpo por entidad

Entidad	Campos del Request Body
Rol	`nombre` (string, requerido, único, máx. 50)
Usuario	`nombre` (string, requerido), `correo` (string, requerido, email, único), `password` (string, requerido solo en creación, mín. 8), `rol_id` (int, requerido, debe existir), `foto` (file, opcional, jpg/png, máx. 2MB)
Log	solo lectura: `usuario_id`, `accion`, `fecha`, `critico` (generados automáticamente por el sistema, no se reciben por request body)

6. Especificación de las respuestas

Todas las respuestas usan **JSON** como formato de intercambio.

Listado (GET colección), paginado:

```
{
  "data": [
    { "id": 15, "nombre": "Chamarra de mezclilla", "precio": 250.00,
"categoria": { "id": 3, "nombre": "Abrigos" }, "usuario": { "id": 5,
"nombre": "David Cohen" }, "imagen": "/img/productos/Producto_15_1.jpg" }
  ],
  "meta": { "current_page": 1, "per_page": 5, "total": 32, "last_page": 7 }
}
```

Consulta de un registro (GET por id): el mismo objeto de `data` anterior, sin envoltura de paginación.

Creación (POST) exitosa — 201 Created:

```
{ "message": "Producto creado correctamente.", "data": { "id": 33, "nombre":
"Chamarra de mezclilla", ... } }
```

Actualización (PUT/PATCH) exitosa — 200 OK: mismo formato que la creación, con `"message": "Producto actualizado correctamente."`.

Eliminación lógica, restauración o eliminación física exitosa — 200 OK (con mensaje) o 204 No Content (sin cuerpo):

```
{ "message": "Producto eliminado correctamente." }
```

Cada campo de `data` corresponde directamente a una columna de la entidad; las relaciones (`categoria`, `usuario`, `rol`) se devuelven como objetos anidados con su descripción, nunca solo el id foráneo, para que el consumidor de la API no tenga que hacer una segunda consulta.

7. Mensajes de error

Todos los errores devuelven `{ "message": "...", "errors": { ... } }` (este último campo solo en errores de validación).

Situación	Mensaje	Código HTTP
Datos inválidos (formato incorrecto, tipo de dato, valor fuera de rango)	"El precio debe ser un valor numérico mayor a 0."	422

Situación	Mensaje	Código HTTP
Campo obligatorio ausente	"El campo nombre es obligatorio."	422
Registro inexistente	"El producto solicitado no existe."	404
Acceso no autorizado (token válido pero sin permiso)	"No tienes permisos para realizar esta acción."	403
Autenticación requerida (sin token o token inválido/expirado)	"Debes iniciar sesión para acceder a este recurso."	401
Permisos insuficientes para una acción específica (ej. eliminar físicamente)	"Tu rol no tiene permiso para eliminar definitivamente este registro."	403
Conflicto de integridad referencial (registro con relaciones dependientes)	"No es posible eliminar: el usuario tiene productos asociados."	409
Error interno del servidor	"Ocurrió un error inesperado. Intenta de nuevo más tarde."	500

8. Códigos de estado HTTP

Código	Significado	Dónde se genera
200 OK	Operación exitosa con contenido de respuesta	GET, PUT, PATCH, DELETE (lógico) exitosos
201 Created	Registro creado correctamente	POST exitoso
204 No Content	Operación exitosa sin cuerpo de respuesta	DELETE físico (forzar) exitoso
400 Bad Request	Solicitud mal formada (JSON inválido, parámetro con tipo incorrecto)	Cualquier endpoint
401 Unauthorized	No se envió token o el token es inválido/expirado	Cualquier endpoint de `/api/admin`

Código	Significado	Dónde se genera
403 Forbidden	Usuario autenticado sin el rol/permiso requerido	Endpoints restringidos por rol (ver sección 10)
404 Not Found	El id solicitado no existe en la base de datos	GET/PUT/PATCH/DELETE por id
409 Conflict	El registro tiene relaciones dependientes y no puede eliminarse físicamente	DELETE /forzar
422 Unprocessable Entity	Error de validación de los datos enviados	POST, PUT, PATCH
500 Internal Server Error	Error no controlado del servidor	Cualquier endpoint

9. Autenticación administrativa

La autenticación se implementa con tokens (Laravel Sanctum). El cliente envía el token recibido en el login dentro del header `Authorization: Bearer {token}` en cada petición a `/api/admin`.

POST /api/admin/auth/login

- **Descripción:** inicia sesión y entrega un token de acceso.
- **Headers:** `Accept: application/json`.
- **Request body:** `correo` (string, requerido), `password` (string, requerido).
- **Respuesta 200 OK:**

```
{ "token": "1|aBcD3fGhIjK...", "usuario": { "id": 5, "nombre": "David Cohen", "rol": "Administrador" } }
```

- **Errores:** 422 si faltan campos, 401 si las credenciales no coinciden ("Correo o contraseña incorrectos.").

GET /api/admin/auth/me

- **Descripción:** devuelve los datos del usuario actualmente autenticado a partir de su token.

- **Headers:** `Authorization: Bearer {token}`.
- **Respuesta 200 OK:** mismo objeto `usuario` del login.
- **Errores:** 401 si el token es inválido o expiró.

POST /api/admin/auth/logout

- **Descripción:** invalida el token actual (cierre de sesión).
- **Headers:** `Authorization: Bearer {token}`.
- **Respuesta 200 OK:** `{ "message": "Sesión cerrada correctamente." }`.
- **Errores:** 401 si no hay sesión activa.

10. Autorización administrativa

El catálogo de roles del proyecto (entidad Rol) define dos roles con acceso al área administrativa: **Administrador** (acceso total) y **Soporte** (consulta y restauración, sin eliminar físicamente ni gestionar roles/usuarios). Todo endpoint bajo `/api/admin` requiere autenticación, salvo el login.

Endpoint	¿Requiere autenticación?	Permiso necesario	Roles autorizados
POST /api/admin/auth/login	No	—	Público (credenciales válidas)
GET /api/admin/auth/me, POST /api/admin/auth/logout	Sí	Sesión activa	Administrador, Soporte
GET /api/admin/roles, /usuarios, /productos, /logs (listar y consultar)	Sí	`ver-registros`	Administrador, Soporte
POST/PUT/PATCH /api/admin/productos	Sí	`gestionar-productos`	Administrador, Soporte
POST/PUT/PATCH /api/admin/roles, /usuarios	Sí	`gestionar-usuarios-roles`	Administrador
DELETE /{id} (eliminar lógico) y /{id}/restaurar	Sí	`gestionar-papelera`	Administrador, Soporte

Endpoint	¿Requiere autenticación?	Permiso necesario	Roles autorizados
DELETE /{id}/forzar (eliminar físico, cualquier entidad)	Sí	`eliminar-definitivo`	Administrador

11. Tabla resumen de la API

Módulo	Endpoint	Método	Descripción	¿Requiere auth?	Permiso requerido	Códigos de respuesta
Auth	/api/admin/auth/login	POST	Inicia sesión y entrega token	No	—	200, 401, 422
Auth	/api/admin/auth/me	GET	Usuario autenticado actual	Sí	Sesión activa	200, 401
Auth	/api/admin/auth/logout	POST	Cierra la sesión	Sí	Sesión activa	200, 401
Roles	/api/admin/roles	GET	Listar roles	Sí	ver-registros	200, 401
Roles	/api/admin/roles/{id}	GET	Consultar un rol	Sí	ver-registros	200, 401, 404
Roles	/api/admin/roles	POST	Crear rol	Sí	gestionar-usuarios-roles	201, 401, 403, 422
Roles	/api/admin/roles/{id}	PUT / PATCH	Actualizar rol (total/parcial)	Sí	gestionar-usuarios-roles	200, 401, 403, 404, 422
Roles	/api/admin/roles/{id}	DELETE	Eliminar lógicamente	Sí	gestionar-papelera	200, 401, 403, 404
Roles	/api/admin/roles/papelera	GET	Listar eliminados	Sí	ver-registros	200, 401
Roles	/api/admin/roles/{id}/restaurar	PATCH	Restaurar rol	Sí	gestionar-papelera	200, 401, 403, 404
Roles	/api/admin/roles/{id}/forzar	DELETE	Eliminar físicamente	Sí	eliminar-definitivo	204, 401, 403, 404, 409
Usuarios	/api/admin/usuarios	GET	Listar usuarios	Sí	ver-registros	200, 401
Usuarios	/api/admin/usuarios/{id}	GET	Consultar un usuario	Sí	ver-registros	200, 401, 404
Usuarios	/api/admin/usuarios	POST	Crear usuario	Sí	gestionar-usuarios-roles	201, 401, 403, 422
Usuarios	/api/admin/usuarios/{id}	PUT / PATCH	Actualizar usuario (total/parcial)	Sí	gestionar-usuarios-roles	200, 401, 403, 404, 422

Módulo	Endpoint	Método	Descripción	¿Requiere auth?	Permiso requerido	Códigos de respuesta
Usuarios	/api/admin/usuarios/{id}	DELETE	Eliminar lógicamente	Sí	gestionar-papelera	200, 401, 403, 404
Usuarios	/api/admin/usuarios/papelera	GET	Listar eliminados	Sí	ver-registros	200, 401
Usuarios	/api/admin/usuarios/{id}/restaurar	PATCH	Restaurar usuario	Sí	gestionar-papelera	200, 401, 403, 404
Usuarios	/api/admin/usuarios/{id}/forzar	DELETE	Eliminar físicamente	Sí	eliminar-definitivo	204, 401, 403, 404, 409
Productos	/api/admin/productos	GET	Listar productos	Sí	ver-registros	200, 401
Productos	/api/admin/productos/{id}	GET	Consultar un producto	Sí	ver-registros	200, 401, 404
Productos	/api/admin/productos	POST	Crear producto	Sí	gestionar-productos	201, 401, 403, 422
Productos	/api/admin/productos/{id}	PUT / PATCH	Actualizar producto (total/parcial)	Sí	gestionar-productos	200, 401, 403, 404, 422
Productos	/api/admin/productos/{id}	DELETE	Eliminar lógicamente	Sí	gestionar-papelera	200, 401, 403, 404
Productos	/api/admin/productos/papelera	GET	Listar eliminados	Sí	ver-registros	200, 401
Productos	/api/admin/productos/{id}/restaurar	PATCH	Restaurar producto	Sí	gestionar-papelera	200, 401, 403, 404
Productos	/api/admin/productos/{id}/forzar	DELETE	Eliminar físicamente	Sí	eliminar-definitivo	204, 401, 403, 404, 409
Logs	/api/admin/logs	GET	Listar bitácora	Sí	ver-registros	200, 401
Logs	/api/admin/logs/{id}	GET	Consultar un log	Sí	ver-registros	200, 401, 404
Logs	/api/admin/logs/{id}	DELETE	Eliminar lógicamente	Sí	gestionar-papelera	200, 401, 403, 404
Logs	/api/admin/logs/papelera	GET	Listar eliminados	Sí	ver-registros	200, 401
Logs	/api/admin/logs/{id}/restaurar	PATCH	Restaurar log	Sí	gestionar-papelera	200, 401, 403, 404
Logs	/api/admin/logs/{id}/forzar	DELETE	Eliminar físicamente	Sí	eliminar-definitivo	204, 401, 403, 404, 409